

Chapter 9: Python Scenario-Based Interview Questions & Staff-Level Grills

Core Learning Objective: Master the tough, scenario-driven interview questions asked by FAANG and top Product-Based Companies. Learn how to diagnose real-world production outages, explain CPython runtime edge cases, and articulate senior engineering trade-offs under pressure.

1. Production Crisis Scenarios

Scenario 1: The Memory Leak in a Celery Background Worker

The Interviewer Asks:

"We have a long-running Python worker processing millions of messages daily. Over 48 hours, its RAM usage steadily balloons from 200 MB to 16 GB until the OS kernel OOM-killer terminates the process. But we used `del` on every processed message! How do you diagnose and eliminate this leak?"

The Staff-Level Answer & Walkthrough:

1. Explain the Root Cause:

`del` only decrements `ob_refcnt` by 1; it does **not** free memory if a circular reference exists or if global registries hold pointers. Furthermore, CPython's PyMalloc allocator does not always return freed memory pages back to the OS—it keeps them in arenas for future allocations (heap fragmentation).

2. Diagnostic Procedure:

- Enable `tracemalloc` to take differential snapshots (`snapshot2.compare_to(snapshot1, 'lineno')`) to isolate which line allocates surviving objects.
- Use `gc.get_objects()` and the `objgraph` library to generate visual back-reference graphs:

```
import objgraph
# Finds what is preventing Garbage Collection of MyTask objects
objgraph.show_backrefs(objgraph.by_type('MyTask')[:1], filename='leak.png')
```

3. Common Culprits & Fixes:

- **Module-level caches / Class variables:** Objects appended to a list/dict that is never pruned.
Fix: Use `weakref.WeakValueDictionary` or bounded LRU caches with explicit TTLs.
- **Circular References with `__del__`:** Prior to Python 3.4, objects with `__del__` in a cycle went to `gc.garbage` uncollected. In modern Python, break cycles using `weakref.ref`.
- **OS Memory Compaction:** Use `malloc_trim(0)` via `ctypes` on Linux to force glibc to release unfragmented pages back to the OS kernel.

Scenario 2: The Event Loop Freeze in an Asyncio Microservice

The Interviewer Asks:

"Our FastAPI microservice is built with `async def` endpoints, designed to handle 5,000 concurrent requests. However, during load testing, when one endpoint performs password hashing or queries a legacy database, all other endpoints freeze and latency spikes to 10 seconds. What is happening?"

The Staff-Level Answer & Walkthrough:

- **Root Cause (Event Loop Starvation):**

`asyncio` operates on a **single OS thread**. An `await` only yields control if the awaited function is a non-blocking asynchronous coroutine. If someone executes a CPU-heavy task (`bcrypt.hashpw`) or a synchronous blocking I/O call (`requests.get()` or synchronous `time.sleep()`), the thread blocks. The event loop cannot tick, starving all 4,999 other concurrent coroutines.

- **The Solution:**

Offload blocking CPU or legacy sync I/O calls to a separate worker thread pool using `asyncio.to_thread()`:

```
import asyncio
import bcrypt

# ANTI-PATTERN (Freezes entire service!):
async def bad_login(password: str, hashed: bytes):
    return bcrypt.checkpw(password.encode(), hashed)

# PRODUCTION PATTERN (Delegates to thread pool; event loop stays free!):
async def good_login(password: str, hashed: bytes):
    return await asyncio.to_thread(bcrypt.checkpw, password.encode(), hashed)
```

Scenario 3: High Concurrency Multithreading Slowdown

The Interviewer Asks:

"We converted an image thumbnail generator from 1 thread to 8 OS threads on an 8-core CPU, expecting an 8x speedup. Instead, the total execution time was 20% SLOWER than the single-threaded version! Why?"

The Staff-Level Answer:

- Image compression/processing in pure Python is a **CPU-bound workload**.
- Due to CPython's **Global Interpreter Lock (GIL)**, only one OS thread can execute Python bytecode at any given millisecond.
- When 8 threads run on 8 physical cores, they aggressively contend for the single GIL mutex. The CPU wastes cycles on OS thread context-switching and lock contention (the "convoy effect"), running slower than a single thread.
- **The Fix:** Switch from `threading.Thread` to `multiprocessing.Pool` or `concurrent.futures.ProcessPoolExecutor`. Each process runs an independent CPython interpreter on its own core with its own GIL.

2. Tricky CPython Runtime & Language Edge Cases

Question 1: "Is `a += b` always identical to `a = a + b`?"

Answer: No!

- `a = a + b` evaluates `a.__add__(b)` and rebinds the name `a` to a **newly allocated object**.

- `a += b` attempts to evaluate `a.__iadd__(b)` for **in-place mutation**.
 - For immutable types (`int` , `str` , `tuple`), both allocate a new object because the underlying data cannot mutate.
 - For mutable types (`list`), `+=` calls `list.extend()` in-place without changing the pointer:

```
x = [1, 2]
y = x
x += [3]      # Mutates in place: y is now [1, 2, 3]!
x = x + [4]   # Allocates new list: y remains [1, 2, 3]!
```

Question 2: "What is the difference between `__getattr__` and `__getattribute__` ?"

Answer:

- `__getattribute__(self, name)` is invoked **unconditionally** on every single attribute access. Calling `self.x` inside it causes infinite recursion; you must call `super().__getattribute__(name)`.
- `__getattr__(self, name)` is a **fallback hook** invoked **only** if the attribute is not found in the instance dictionary or its class hierarchy.

Question 3: "Why does Python have `__slots__` if it already has dictionaries?"

Answer: By default, every Python instance stores attributes in a dynamic dictionary (`self.__dict__`), which incurs substantial memory overhead (indices array, entries array, hash table padding: ~150-200 bytes per instance).

`__slots__` replaces `__dict__` with a **static C struct array of pointers**, saving \$>68%\$ of RAM per instance and speeding up attribute access by bypassing dictionary lookups.